

# Module 12 Notes: Multi-List Operations — Concatenation and Merge

## 1. Current Progress Analysis

You are ready for **Module 12** because you already know the important pointer skills needed for this module.

Module 12 is not about a completely new linked-list idea. It is mainly about using the ideas you already learned with **two linked lists** instead of only one.

### What you already know

| Previous Module | Skill Needed for Module 12                                                                                           |
|-----------------|----------------------------------------------------------------------------------------------------------------------|
| Module 2        | A node has <code>data</code> and <code>next</code> . The <code>next</code> part stores the address of the next node. |
| Module 3        | Traversal means moving through a list using <code>p = p-&gt;next</code> .                                            |
| Module 6        | Searching uses pointer movement through nodes.                                                                       |
| Module 7        | Insertion taught how to change links safely.                                                                         |
| Module 8        | <code>last</code> can be used to attach nodes at the end.                                                            |
| Module 9        | Sorted linked lists must keep values in increasing order.                                                            |
| Module 10       | <code>p</code> and <code>q</code> are used to connect or skip nodes.                                                 |
| Module 11       | Reversal showed how to change links without creating new nodes.                                                      |

### Why these skills matter now

In Module 12, we work with **more than one linked list**.

Instead of only having:

```
first -> 10 -> 20 -> 30 -> NULL
```

we may have:

```
first  -> 2 -> 9 -> 6 -> 8 -> NULL
second -> 10 -> 15 -> NULL
```

So Module 12 teaches how to combine linked lists.

## 2. Big Idea of Module 12

Module 12 teaches two main multi-list operations:

1. **Concatenation**
2. **Merge**

These two operations both combine lists, but they do not mean the same thing.

| Operation     | Simple Meaning                                        |
|---------------|-------------------------------------------------------|
| Concatenation | Attach the second list directly after the first list. |
| Merge         | Combine two sorted lists into one sorted list.        |

### Important difference

Concatenation does **not** care about sorted order.

Merge does care about sorted order.

Example:

```
Concatenation:
2 -> 9 -> 6 -> 8 + 10 -> 15

Result:
2 -> 9 -> 6 -> 8 -> 10 -> 15
```

But merge works like this:

```
Merge:
2 -> 8 -> 10 + 4 -> 7

Result:
2 -> 4 -> 7 -> 8 -> 10
```

# Part A: Concatenation

---

## 3. What Is Concatenation?

Concatenation means:

Join the second linked list directly after the first linked list.

Example:

```
first  -> 2 -> 9 -> 6 -> 8 -> NULL
second -> 10 -> 15 -> NULL
```

After concatenation:

```
first -> 2 -> 9 -> 6 -> 8 -> 10 -> 15 -> NULL
```

The second list is now attached to the first list.

So after concatenation, the combined list can be accessed using `first`.

---

## 4. Main Idea Behind Concatenation

To concatenate two lists, we need to connect the last node of the first list to the first node of the second list.

Given:

```
first  -> 2 -> 9 -> 6 -> 8 -> NULL
second -> 10 -> 15 -> NULL
```

The last node of the first list is:

```
8
```

The first node of the second list is:

```
10
```

So we must change:

```
8 -> NULL
```

into:

```
8 -> 10
```

In code, that means:

```
p->next = second;
```

where `p` is pointing to the last node of the first list.

---

## 5. Why We Must Stop at the Last Node

To concatenate correctly, pointer `p` must stop on the **last node** of the first list.

Example:

```
2 -> 9 -> 6 -> 8 -> NULL
```

We want `p` to stop here:

```
2 -> 9 -> 6 -> 8 -> NULL
                ^
                |
                p
```

Then we can write:

```
p->next = second;
```

This changes the last node's `next` from `NULL` to the start of the second list.

---

## 6. Correct Loop for Finding the Last Node

The correct loop is:

```
while (p->next != NULL) {  
    p = p->next;  
}
```

### Why this works

The condition checks:

```
p->next != NULL
```

This means:

Keep moving while there is another node after `p`.

When `p->next == NULL`, `p` is pointing to the last node.

---

## 7. Wrong Loop for Concatenation

This loop is wrong for concatenation:

```
while (p != NULL) {  
    p = p->next;  
}
```

Why?

Because this loop continues until `p` becomes `NULL`.

At the end:

```
p == NULL
```

Then this is invalid:

```
p->next = second;
```

because `p` is not pointing to a real node.

### Beginner rule

For concatenation:

Stop on the last node, not after the last node.

So use:

```
while (p->next != NULL)
```

not:

```
while (p != NULL)
```

---

## 8. Concatenation Code in C

```
void Concatenate(struct Node *p, struct Node *q) {  
    if (p == NULL) {  
        first = q;  
        second = NULL;  
        return;  
    }  
  
    if (q == NULL) {  
        return;  
    }  
  
    while (p->next != NULL) {  
        p = p->next;  
    }  
  
    p->next = q;  
    second = NULL;  
}
```

Usually we call it like this:

```
Concatenate(first, second);
```

## 9. Explanation of Concatenation Code

### Function header

```
void Concatenate(struct Node *p, struct Node *q)
```

Meaning:

| Parameter | Meaning                                      |
|-----------|----------------------------------------------|
| p         | Pointer used to move through the first list. |
| q         | Pointer to the beginning of the second list. |

The function returns `void` because it directly changes links in the list.

### Empty first list case

```
if (p == NULL) {  
    first = q;  
    second = NULL;  
    return;  
}
```

If the first list is empty, then the final combined list is just the second list.

Example:

```
first  -> NULL  
second -> 10 -> 15 -> NULL
```

After concatenation:

```
first -> 10 -> 15 -> NULL
```

So we set:

```
first = q;
```

Then:

```
second = NULL;
```

because the second list is no longer separate.

---

### Empty second list case

```
if (q == NULL) {  
    return;  
}
```

If the second list is empty, there is nothing to attach.

Example:

```
first  -> 2 -> 9 -> NULL  
second -> NULL
```

After concatenation, the first list stays the same:

```
first -> 2 -> 9 -> NULL
```

---

### Find the last node of the first list

```
while (p->next != NULL) {  
    p = p->next;  
}
```

This moves `p` until it reaches the last node of the first list.

---



### Attach the second list

```
p->next = q;
```

This connects the last node of the first list to the first node of the second list.

### Remove separate access through `second`

```
second = NULL;
```

After concatenation, the second list is now part of the first list.

So we can optionally set `second` to `NULL` to show that it is no longer a separate list.

## 10. Dry Run: Concatenation

Given:

```
first  -> 2 -> 9 -> 6 -> 8 -> NULL
second -> 10 -> 15 -> NULL
```

Call:

```
Concatenate(first, second);
```

Start:

```
p -> 2
q -> 10
```

Move `p` until `p->next == NULL`.

| Step  | <code>p</code> points to | <code>p-&gt;next</code> points to | Action        |
|-------|--------------------------|-----------------------------------|---------------|
| Start | 2                        | 9                                 | Move forward. |
| 1     | 9                        | 6                                 | Move forward. |
| 2     | 6                        | 8                                 | Move forward. |

| Step | p points to | p->next points to | Action                       |
|------|-------------|-------------------|------------------------------|
| 3    | 8           | NULL              | Stop. This is the last node. |

Now p points to 8.

Do:

```
p->next = q;
```

Since q points to 10, this means:

```
8->next = 10
```

Final list:

```
first -> 2 -> 9 -> 6 -> 8 -> 10 -> 15 -> NULL
```

Then:

```
second = NULL;
```

Now the combined list is accessed through first.

## 11. Concatenation Complexity

Let the first list have n nodes.

To concatenate normally, we must travel from the first node to the last node of the first list.

So:

```
Time complexity: O(n)
```

The operation only uses a few pointer variables.

So:

Extra space complexity:  $O(1)$

### Important optimization idea

If we already had a `last` pointer for the first list, concatenation could be done faster.

With a `last` pointer:

```
last->next = second;
```

This would be:

$O(1)$

But in the normal version, we do not assume we already have `last`, so we traverse the first list.

---

## Part B: Merge

---

### 12. What Is Merge?

Merge means:

Combine two sorted linked lists into one sorted linked list.

Example:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

After merging:

```
third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL
```

The final list is sorted.

---

## 13. Important Rule for Merge

Merge assumes both input lists are already sorted.

Correct input:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

Both lists are sorted.

Wrong input:

```
first  -> 8 -> 2 -> 10 -> NULL
second -> 7 -> 4 -> NULL
```

These lists are not sorted.

If the input lists are not sorted, the merge function will not produce a correctly sorted final list.

---

## 14. Merge Does Not Create New Nodes

This is one of the most important ideas in Module 12.

Merge does **not** create new nodes for every value.

It does not do this repeatedly:

```
malloc(...)
```

Instead, merge reuses the existing nodes from the first and second lists.

It only changes links.

### Example

Before:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

The nodes already exist.

Merge rearranges links to form:

```
third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL
```

No new data nodes are needed.

---

## 15. Pointers Used in Merge

Merge uses several pointers.

| Pointer             | Meaning                                             |
|---------------------|-----------------------------------------------------|
| <code>first</code>  | Points to the start of the first sorted list.       |
| <code>second</code> | Points to the start of the second sorted list.      |
| <code>third</code>  | Points to the start of the final merged list.       |
| <code>p</code>      | Moving pointer in the first list.                   |
| <code>q</code>      | Moving pointer in the second list.                  |
| <code>last</code>   | Points to the current last node of the merged list. |

### Simple way to remember

```
third = head of the merged list
last  = tail of the merged list
p     = current node in first list
q     = current node in second list
```

---

## 16. Why We Need `third`

When we merge two lists, we are creating one final combined chain.

We need a pointer to the first node of that final chain.

That pointer is called:

```
third
```

Example final result:

```
third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL
```

Without `third`, we would not know where the merged list begins.

---

## 17. Why We Need `last`

While building the merged list, we keep adding nodes at the end.

So we need a pointer to the current end of the merged list.

That pointer is called:

```
last
```

Example:

```
third -> 2 -> 4 -> NULL
          ^
          |
        last
```

If the next smaller node is `7`, we can attach it after `last`.

```
last->next = q;
last = q;
```

Then:

```
third -> 2 -> 4 -> 7 -> NULL
                  ^
```

|  
last

## 18. Main Merge Logic in Simple Words

Suppose:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

We compare the first available node from both lists.

### Step 1

Compare:

2 and 4

Smaller value is 2.

So the merged list starts with 2.

```
third -> 2 -> NULL
```

### Step 2

Now compare:

8 and 4

Smaller value is 4.

Attach 4 after 2.

```
third -> 2 -> 4 -> NULL
```

### Step 3

Now compare:

8 and 7

Smaller value is 7.

Attach 7 after 4.

third -> 2 -> 4 -> 7 -> NULL

#### Step 4

Now the second list is finished.

So attach the remaining part of the first list:

8 -> 10 -> NULL

Final merged list:

third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL

---

## 19. Merge Code in C

```
void Merge(struct Node *p, struct Node *q) {
    struct Node *last;

    if (p == NULL) {
        third = q;
        return;
    }

    if (q == NULL) {
        third = p;
        return;
    }

    if (p->data < q->data) {
        third = last = p;
        p = p->next;
        last->next = NULL;
    } else {
```



```

        third = last = q;
        q = q->next;
        last->next = NULL;
    }

    while (p != NULL && q != NULL) {
        if (p->data < q->data) {
            last->next = p;
            last = p;
            p = p->next;
            last->next = NULL;
        } else {
            last->next = q;
            last = q;
            q = q->next;
            last->next = NULL;
        }
    }

    if (p != NULL) {
        last->next = p;
    } else {
        last->next = q;
    }
}

```

Call:

```
Merge(first, second);
```

Display the merged list:

```
display(third);
```

## 20. Explanation of Merge Code

### Function header

```
void Merge(struct Node *p, struct Node *q)
```

Meaning:

| Parameter | Meaning                     |
|-----------|-----------------------------|
| p         | Pointer to the first list.  |
| q         | Pointer to the second list. |

---

**Local pointer** `last`

```
struct Node *last;
```

`last` points to the final node currently in the merged list.

We use it to attach the next smallest node.

---

## 21. Empty List Cases in Merge

### Case 1: First list is empty

```
if (p == NULL) {  
    third = q;  
    return;  
}
```

If the first list is empty, then the merged list is just the second list.

Example:

```
first  -> NULL  
second -> 4 -> 7 -> NULL
```

Merged result:

```
third -> 4 -> 7 -> NULL
```

---

### Case 2: Second list is empty

```
if (q == NULL) {  
    third = p;
```

```
    return;  
}
```

If the second list is empty, then the merged list is just the first list.

Example:

```
first  -> 2 -> 8 -> 10 -> NULL  
second -> NULL
```

Merged result:

```
third -> 2 -> 8 -> 10 -> NULL
```

---

## 22. Initial Setup of Merge

Before the main loop, we must choose the first node of the merged list.

We compare:

```
p->data  
q->data
```

If `p->data` is smaller:

```
third = last = p;  
p = p->next;  
last->next = NULL;
```

Otherwise:

```
third = last = q;  
q = q->next;  
last->next = NULL;
```

### Why this is needed

Before using `last->next`, we need `last` to point to a real node.

So we first choose the first node of the merged list.

---

## 23. Why We Set `last->next = NULL`

This line may look confusing:

```
last->next = NULL;
```

But it is important.

Suppose we choose node `2` from this list:

```
2 -> 8 -> 10 -> NULL
```

Originally, `2` points to `8`.

But in the merged list, the next node after `2` might not be `8`.

It might be `4` from the second list.

So we temporarily detach `2`:

```
2 -> NULL
```

Then later, we attach the correct next node.

This helps build the merged list cleanly.

---

## 24. Main Merge Loop

The loop is:

```
while (p != NULL && q != NULL) {  
    if (p->data < q->data) {  
        last->next = p;  
        last = p;  
        p = p->next;  
        last->next = NULL;  
    } else {
```

```

        last->next = q;
        last = q;
        q = q->next;
        last->next = NULL;
    }
}

```

The loop continues while both lists still have nodes.

**Why use** `p != NULL && q != NULL` ?

Because we can only compare:

```

p->data
q->data

```

when both `p` and `q` point to real nodes.

If one list ends, we stop comparing and attach the remaining part of the other list.

## 25. If `p` Has the Smaller Value

Code:

```

last->next = p;
last = p;
p = p->next;
last->next = NULL;

```

Meaning:

| Line                               | Meaning                                                   |
|------------------------------------|-----------------------------------------------------------|
| <code>last-&gt;next = p;</code>    | Attach node <code>p</code> after the current merged list. |
| <code>last = p;</code>             | Move <code>last</code> to the newly attached node.        |
| <code>p = p-&gt;next;</code>       | Move <code>p</code> forward in the first list.            |
| <code>last-&gt;next = NULL;</code> | Detach the attached node from its old forward link.       |

## 26. If `q` Has the Smaller Value

Code:

```
last->next = q;  
last = q;  
q = q->next;  
last->next = NULL;
```

Meaning:

| Line                               | Meaning                                                   |
|------------------------------------|-----------------------------------------------------------|
| <code>last-&gt;next = q;</code>    | Attach node <code>q</code> after the current merged list. |
| <code>last = q;</code>             | Move <code>last</code> to the newly attached node.        |
| <code>q = q-&gt;next;</code>       | Move <code>q</code> forward in the second list.           |
| <code>last-&gt;next = NULL;</code> | Detach the attached node from its old forward link.       |

## 27. Attaching the Remaining Nodes

When one list becomes empty, the loop stops.

Example:

```
p -> 8 -> 10 -> NULL  
q -> NULL
```

There is no need to compare anymore.

The rest of `p` is already sorted.

So attach it directly:

```
if (p != NULL) {  
    last->next = p;  
} else {  
    last->next = q;  
}
```

## Why this is safe

Both original lists were sorted.

So if one list ends, the remaining nodes of the other list are already in correct order.

---

## 28. Dry Run: Merge

Given:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

Call:

```
Merge(first, second);
```

Initial:

```
p -> 2
q -> 4
```

---

### Step 1: Choose first node

Compare:

```
2 and 4
```

**2** is smaller.

So:

```
third -> 2 -> NULL
last  -> 2
p      -> 8
q      -> 4
```

---

### Step 2: Compare 8 and 4

4 is smaller.

Attach 4 after last .

```
third -> 2 -> 4 -> NULL
last  -> 4
p      -> 8
q      -> 7
```

---

### Step 3: Compare 8 and 7

7 is smaller.

Attach 7 after last .

```
third -> 2 -> 4 -> 7 -> NULL
last  -> 7
p      -> 8
q      -> NULL
```

---

### Step 4: Second list ended

Now:

```
q == NULL
```

So attach the remaining first list:

```
8 -> 10 -> NULL
```

Final result:

```
third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL
```

---



## 29. Merge Complexity

Let:

```
m = number of nodes in first list  
n = number of nodes in second list
```

Merge may visit every node from both lists once.

So:

```
Time complexity:  $O(m + n)$ 
```

Merge only uses pointer variables like `p`, `q`, `third`, and `last`.

So:

```
Extra space complexity:  $O(1)$ 
```

Important:

This does not count the original nodes, because those nodes already exist.

---

## Part C: Full Example Program

### 30. Full Working C Program for Merge

```
#include <stdio.h>  
#include <stdlib.h>  
  
struct Node {  
    int data;  
    struct Node *next;  
};  
  
struct Node *first = NULL;  
struct Node *second = NULL;  
struct Node *third = NULL;
```

```

void createFirst(int A[], int n) {
    struct Node *t, *last;

    if (n <= 0) {
        first = NULL;
        return;
    }

    first = (struct Node *)malloc(sizeof(struct Node));
    first->data = A[0];
    first->next = NULL;
    last = first;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;
        last->next = t;
        last = t;
    }
}

void createSecond(int A[], int n) {
    struct Node *t, *last;

    if (n <= 0) {
        second = NULL;
        return;
    }

    second = (struct Node *)malloc(sizeof(struct Node));
    second->data = A[0];
    second->next = NULL;
    last = second;

    for (int i = 1; i < n; i++) {
        t = (struct Node *)malloc(sizeof(struct Node));
        t->data = A[i];
        t->next = NULL;
        last->next = t;
        last = t;
    }
}

void display(struct Node *p) {
    while (p != NULL) {
        printf("%d ", p->data);
        p = p->next;
    }
}

```

```

    }
}

void Concatenate(struct Node *p, struct Node *q) {
    if (p == NULL) {
        first = q;
        second = NULL;
        return;
    }

    if (q == NULL) {
        return;
    }

    while (p->next != NULL) {
        p = p->next;
    }

    p->next = q;
    second = NULL;
}

void Merge(struct Node *p, struct Node *q) {
    struct Node *last;

    if (p == NULL) {
        third = q;
        return;
    }

    if (q == NULL) {
        third = p;
        return;
    }

    if (p->data < q->data) {
        third = last = p;
        p = p->next;
        last->next = NULL;
    } else {
        third = last = q;
        q = q->next;
        last->next = NULL;
    }

    while (p != NULL && q != NULL) {
        if (p->data < q->data) {
            last->next = p;

```

```

        last = p;
        p = p->next;
        last->next = NULL;
    } else {
        last->next = q;
        last = q;
        q = q->next;
        last->next = NULL;
    }
}

if (p != NULL) {
    last->next = p;
} else {
    last->next = q;
}
}

int main() {
    int A[] = {2, 8, 10};
    int B[] = {4, 7};

    createFirst(A, 3);
    createSecond(B, 2);

    printf("First list: ");
    display(first);

    printf("\nSecond list: ");
    display(second);

    Merge(first, second);

    printf("\nMerged list: ");
    display(third);

    return 0;
}

```

Expected output:

```

First list: 2 8 10
Second list: 4 7
Merged list: 2 4 7 8 10

```

## Part D: Concatenation vs Merge

### 31. Main Difference Between Concatenation and Merge

| Feature                        | Concatenation                                          | Merge                                                                        |
|--------------------------------|--------------------------------------------------------|------------------------------------------------------------------------------|
| Meaning                        | Attach second list after first list.                   | Combine two sorted lists into one sorted list.                               |
| Input lists need to be sorted? | No.                                                    | Yes.                                                                         |
| Final list sorted?             | Not necessarily.                                       | Yes, if both inputs were sorted.                                             |
| Main action                    | Find last node of first list, then attach second list. | Compare nodes one by one and attach the smaller node.                        |
| Uses <code>third</code> ?      | Usually no.                                            | Yes.                                                                         |
| Creates new nodes?             | No.                                                    | No.                                                                          |
| Changes links?                 | Yes.                                                   | Yes.                                                                         |
| Time complexity                | $O(n)$ where <code>n</code> is the first list length.  | $O(m + n)$ where <code>m</code> and <code>n</code> are the two list lengths. |
| Extra space                    | $O(1)$                                                 | $O(1)$                                                                       |

### 32. Simple Example Comparing Both

#### Concatenation example

Input:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

After concatenation:

```
first -> 2 -> 8 -> 10 -> 4 -> 7 -> NULL
```

This result is not sorted.

## Merge example

Input:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

After merge:

```
third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL
```

This result is sorted.

---

## Part E: Common Beginner Mistakes

---

### 33. Mistake 1: Using the Wrong Loop in Concatenation

Wrong:

```
while (p != NULL) {
    p = p->next;
}

p->next = second;
```

Why this is wrong:

At the end of the loop, `p` is `NULL`.

So this is unsafe:

```
p->next
```

Correct:

```
while (p->next != NULL) {
    p = p->next;
}
```

```
p->next = second;
```

## 34. Mistake 2: Thinking Merge Creates New Nodes

Merge does not create new nodes for each value.

Wrong idea:

```
Create a new node for 2.  
Create a new node for 4.  
Create a new node for 7.  
...
```

Correct idea:

```
Reuse existing nodes and change their links.
```

## 35. Mistake 3: Trying to Merge Unsorted Lists

Merge only works correctly when both lists are sorted.

Good:

```
first  -> 2 -> 8 -> 10 -> NULL  
second -> 4 -> 7 -> NULL
```

Bad:

```
first  -> 8 -> 2 -> 10 -> NULL  
second -> 7 -> 4 -> NULL
```

If the lists are not sorted, the output will not be properly sorted.

## 36. Mistake 4: Forgetting to Display `third` After Merge

After merge, the final list starts at:

```
third
```

So display:

```
display(third);
```

Do not assume the final merged list should always be displayed using `first`.

---

## 37. Mistake 5: Forgetting to Attach the Remaining Nodes

In merge, once one list becomes `NULL`, the other list may still have nodes.

Example:

```
third -> 2 -> 4 -> 7 -> NULL
p      -> 8 -> 10 -> NULL
q      -> NULL
```

If you forget to attach `p`, the final list becomes incomplete.

Correct:

```
if (p != NULL) {
    last->next = p;
} else {
    last->next = q;
}
```

## 38. Mistake 6: Confusing Concatenation With Merge

Concatenation simply joins lists.

Merge compares values.



## Concatenation

Attach list 2 after list 1.

## Merge

Compare values and build sorted result.

---

# Part F: Edge Cases

---

## 39. Edge Cases for Concatenation

### Case 1: First list is empty

```
first  -> NULL
second -> 10 -> 20 -> NULL
```

Result:

```
first -> 10 -> 20 -> NULL
```

---

### Case 2: Second list is empty

```
first  -> 5 -> 9 -> NULL
second -> NULL
```

Result:

```
first -> 5 -> 9 -> NULL
```

---

### Case 3: Both lists are empty

```
first  -> NULL  
second -> NULL
```

Result:

```
first -> NULL
```

---

## 40. Edge Cases for Merge

### Case 1: First list is empty

```
first  -> NULL  
second -> 4 -> 7 -> NULL
```

Result:

```
third -> 4 -> 7 -> NULL
```

---

### Case 2: Second list is empty

```
first  -> 2 -> 8 -> NULL  
second -> NULL
```

Result:

```
third -> 2 -> 8 -> NULL
```

---

### Case 3: Both lists are empty

```
first  -> NULL  
second -> NULL
```

Result:

```
third -> NULL
```

---

#### Case 4: One list finishes earlier

Input:

```
first  -> 2 -> 8 -> 10 -> 12 -> NULL
second -> 4 -> NULL
```

After comparing and attaching **2** and **4**, the second list ends.

Remaining first list:

```
8 -> 10 -> 12 -> NULL
```

Attach it directly.

Final:

```
third -> 2 -> 4 -> 8 -> 10 -> 12 -> NULL
```

---

## Part G: Final Summary

---

### 41. Module 12 Final Summary

Module 12 teaches how to work with **two linked lists**.

The two important operations are:

1. **Concatenation**
  2. **Merge**
-

## 42. Concatenation Summary

Concatenation means:

Join list 2 after list 1.

Example:

```
first  -> 2 -> 9 -> 6 -> NULL
second -> 10 -> 15 -> NULL
```

Result:

```
first -> 2 -> 9 -> 6 -> 10 -> 15 -> NULL
```

Key code:

```
while (p->next != NULL) {
    p = p->next;
}

p->next = second;
second = NULL;
```

Time complexity:

$O(n)$

Extra space:

$O(1)$

---

## 43. Merge Summary

Merge means:

Combine two sorted lists into one sorted list.

Example:

```
first  -> 2 -> 8 -> 10 -> NULL
second -> 4 -> 7 -> NULL
```

Result:

```
third -> 2 -> 4 -> 7 -> 8 -> 10 -> NULL
```

Key idea:

```
Compare p->data and q->data.
Attach the smaller node to last.
Move that pointer forward.
Repeat until one list ends.
Attach the remaining nodes.
```

Key pointers:

```
third = start of merged list
last  = end of merged list
p      = current node in first list
q      = current node in second list
```

Time complexity:

$O(m + n)$

Extra space:

$O(1)$

---

## 44. Most Important Lesson

The most important lesson of Module 12 is:

Concatenation attaches whole lists.  
Merge compares nodes one by one.  
Both operations change links, not data.

Concatenation is simple joining.

Merge is sorted combining.

Both are powerful because they reuse existing linked-list nodes instead of shifting array elements or copying everything into a new structure.